

ALLIANZ LABUBU



Shaman Alonso Amezcua
Keneth Sebastian Campos
Marcos Cobo Gutiérrez
Gabriel Gutiérrez Portal
Aritz de la Pinta Morales
Alexander Díez Ortuzar

Índice

Índice.....	1
Anti-clickjacking.....	2
Líneas específicas (anti-clickjacking).....	2
Explicación:.....	2
Donde colocar las líneas:.....	2
Ejemplo en login_form.php:.....	2
SameSite y HttpOnly.....	3
Explicación:.....	3
Donde colocar las líneas:.....	3
Ejemplo en login_form.php:.....	3
CSRF (Cross-Site Request Forgery).....	5
Líneas específicas (protección CSRF).....	5
En el formulario HTML:.....	5
Explicación:.....	5
Dónde colocar las líneas:.....	5
Ejemplo en login_form.php:.....	6
CSP.....	7
Explicación:.....	7
Cabecera.....	7
Ejemplo login_form.php:.....	7
X-Powered-By.....	8
Explicación:.....	8
X-Content-Type-Options.....	9
Explicación:.....	9
Ejemplo en .htaccess en /app:.....	9
Ejemplo en docker-compose.yml:.....	9
Comentarios sospechosos.....	10
Explicación:.....	10
Ataques por fuerza bruta.....	11
Explicación:.....	11
Ocultar cabecera “Server”.....	12
Explicación:.....	12
Donde colocar las líneas:.....	12
Bibliografía.....	13

Anti-clickjacking

Líneas específicas (anti-clickjacking)

```
header("X-Frame-Options: SAMEORIGIN");
```

```
header("Content-Security-Policy: frame-ancestors 'self'");
```

- X-Frame-Options: SAMEORIGIN evita que la página se muestre dentro de un iframe.
- X-Content-Type-Options: nosniff evita que el navegador adivine el tipo de archivo y protege contra scripts maliciosos.

Explicación:

Ambas cabeceras impiden que la página sea incrustada por terceros en <iframe> u objetos similares. X-Frame-Options ofrece compatibilidad con navegadores antiguos; Content-Security-Policy: frame-ancestors es la política moderna y más completa. Usar las dos proporciona mayor compatibilidad y seguridad.

Donde colocar las líneas:

Colocar estas cabeceras antes de cualquier salida al navegador (antes de session_start() y de cualquier HTML) para que se envíen correctamente.

Ejemplo en login_form.php:

```
<?php
header("Content-Security-Policy: default-src 'self'; script-src 'self' https://code.jquery.com;
style-src 'self' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src
'self' data:; connect-src 'self'; object-src 'none'; frame-ancestors 'none'; base-uri 'self';
form-action 'self';");
// Evitar clickjacking
header("X-Frame-Options: SAMEORIGIN");
header("X-Content-Type-Options: nosniff");

// Eliminar información de versión
header_remove("X-Powered-By");

// Activar sesiones
session_start();
```

SameSite y HttpOnly

Para solucionar esta alerta, hemos aplicado este código en los php de login, register e index:

```
if (PHP_VERSION_ID < 70300) {  
    $params = session_get_cookie_params();  
    $sessionId = session_id();  
    $cookie = sprintf(  
        'PHPSESSID=%s; Path=%s; HttpOnly; SameSite=Lax',  
        $sessionId,  
        $params['path']  
    );  
    if (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') {  
        $cookie .= '; Secure';  
    }  
    header('Set-Cookie: ' . $cookie, true);  
}
```

Explicación:

- Se introduce de manera manual comprobándose que la versión de PHP es menor que 7.3.0, porque antes de esa versión no se podía establecer SameSite directamente usando funciones nativas.
- `$params = session_get_cookie_params();`: se usa para recuperar el path y mantenerlo igual cuando se construya la nueva cookie.
- `$sessionId = session_id();`: recupera el id de la sesión actual para asignarlo a la cookie PHPSESSID que PHP usa por defecto para identificar la sesión del usuario.

Después, se crea una cadena de texto con el formato de cabecera Set-Cookie añadiendo el valor y el path anteriores. Además, se marca httponly para impedir que JavaScript acceda a la cookie y así protegerse contra ataques XSS, y se asigna Samesite=Lax que restringe el envío de la cookie en solicitudes de terceros aumentando la seguridad contra ataques CSRF.

Se añade también la opción de verificar primero si la conexión es https y si es así, añade Secure para evitar que la cookie se envíe a través de http. Aunque, de momento no se usa porque nuestra web usa http únicamente.

Donde colocar las líneas:

Colocar este código después de cualquier `session_start()` para que se funcione correctamente.

Ejemplo en login.form.php:

```

<?php
header("Content-Security-Policy: default-src 'self'; script-src 'self' https://code.jquery.com;
style-src 'self' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src
'self' data:; connect-src 'self'; object-src 'none'; frame-ancestors 'none'; base-uri 'self';
form-action 'self;");
// Evitar clickjacking
header("X-Frame-Options: SAMEORIGIN");
header('X-Content-Type-Options: nosniff');

// Eliminar información de versión
header_remove("X-Powered-By");

// Activar sesiones
session_start();
//Atributo samesite y httponly activado
if (PHP_VERSION_ID < 70300) {
    $params = session_get_cookie_params();
    $sessionId = session_id();
    $cookie = sprintf(
        'PHPSESSID=%s; Path=%s; HttpOnly; SameSite=Lax',
        $sessionId,
        $params['path']
    );
    if (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') {
        $cookie .= '; Secure';
    }
    header('Set-Cookie: ' . $cookie, true);
}

```

CSRF (Cross-Site Request Forgery)

Líneas específicas (protección CSRF)

```
// Crear token CSRF si no existe
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
$csrf_token = $_SESSION['csrf_token'];

// Validar token CSRF en peticiones POST
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
        die('Error: token CSRF inválido');
    }
}

// Regenerar sesión y token tras iniciar sesión o completar acciones sensibles
session_regenerate_id(true);
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

En el formulario HTML:

```
<input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($csrf_token);
?>">
```

Explicación:

El ataque CSRF consiste en que un tercero intenta enviar formularios en nombre del usuario sin que este lo sepa, aprovechando que el navegador envía automáticamente las cookies de sesión. Para evitarlo, se genera un token único y aleatorio asociado a la sesión del usuario. Ese token se inserta en el formulario legítimo y se envía junto con la petición. Cuando el servidor recibe un POST, compara el token recibido con el que está guardado en la sesión. Si no coinciden, se bloquea la acción. De esta manera, un atacante no puede falsificar una petición porque no tiene acceso al token real, que no está disponible fuera de la sesión del usuario.

Dónde colocar las líneas:

El token debe generarse después de `session_start()`, ya que se almacena en la sesión del usuario. La validación debe hacerse siempre antes de procesar cualquier dato enviado por

POST. La regeneración del token es recomendable hacerla después de acciones sensibles como el inicio de sesión o el envío de formularios importantes.

Dónde colocar las líneas:

El token debe generarse justo después de `session_start()`, y validarse al inicio de cualquier POST antes de procesar datos. La regeneración del token se hace tras completar una acción sensible, como un login correcto.

Ejemplo en `login_form.php`:

```
//Crear token CSRF si no existe
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
$csrf_token = $_SESSION['csrf_token'];
```

[...]

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
        die('Error: token CSRF inválido');
    }
}
```

[...]

```
// Regenerar sesión y token CSRF para seguridad
session_regenerate_id(true);
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

[...]

```
<input type="hidden" name="csrf_token" value="<?php echo
htmlspecialchars($csrf_token); ?>">
```

CSP

Para solucionar esta alerta, hemos aplicado esta cabecera en los php de login, register e index:

```
header("Content-Security-Policy: default-src 'self';  
      script-src 'self' https://code.jquery.com;  
      style-src 'self' https://fonts.googleapis.com;  
      font-src 'self' https://fonts.gstatic.com;  
      img-src 'self' data;;  
      connect-src 'self';  
      object-src 'none';  
      frame-ancestors 'none';  
      base-uri 'self';  
      form-action 'self';");
```

Asimismo, se han creado robots.txt y sitemap.xml con la configuración mínima.

Explicación:

Cabecera

- default-src 'self': es la política por defecto para todos los tipos de recurso. 'self' significa que solo permite recursos del mismo dominio que nuestro sitio.
- script-src 'self' https://code.jquery.com: define dónde se puede cargar scripts js, en este caso, 'self' solo en el mismo dominio que nuestro sitio. 'https://code.jquery.com' permite jQuery desde CDN, es decir, previene que se ejecuten scripts de fuentes no fiables.
- style-src 'self' https://fonts.googleapis.com: para saber de dónde puede cargar estilo CSS, permite local y también de Google Fonts
- font-src 'self' https://fonts.gstatic.com: para saber de dónde puede cargar fuentes tipográficas, permite local y también de Google Fonts
- img-src 'self' data: para imágenes, locales y embebidas como base64 en HTML o CSS
- connect-src 'self': para orígenes permitidos para conexiones AJAX/fetch/WebSockets; solo en nuestro dominio se pueden recibir solicitudes de datos.
- object-src 'none': prohíbe totalmente los elementos <object>, <embed> y <applet>, que suelen ser vectores de ataque.
- frame-ancestors 'none': nadie puede meter nuestra página en un iframe.
- base-uri 'self': restringe <base>, URLs relativas, a nuestro propio dominio.
- form-action 'self': los formularios <form> solo pueden enviarse a nuestro dominio.

Ejemplo login_form.php:

```
// Eliminar información de versión  
header_remove("X-Powered-By");
```


X-Powered-By

Para solucionar este problema, tan solo se ha tenido que incluir la siguiente línea de código en los archivos .php que daban este problema:

```
header_remove("X-Powered-By");
```

Explicación:

Este problema de seguridad viene de que en diferentes direcciones informan de la versión de php que usan mediante el encabezado "X-Powered-By". Para eliminar esa cabecera, usamos el comando *header_remove* con el nombre de la cabecera como parámetro.

X-Content-Type-Options

Este problema se soluciona introduciendo en el código php el siguiente comando:

```
header('X-Content-Type-Options: nosniff');
```

Asimismo, modificando el .htaccess dentro de /app/ y el docker-compose.yml.

Explicación:

El problema se da porque los navegadores pueden saber el tipo de contenido que puede tener una respuesta. Añadiendo una cabecera mediante *header()* escribiendo la opción de X-Content-Type a *nosniff*, podemos hacer que los navegadores no puedan rastrear el tipo de contenido.

En el caso del .htaccess, comprueba si tiene el módulo mod_headers de Apache cargado (<IfModule mod_headers.c>), previene de ataques que intentan adivinar el tipo MIME y solo interpreta el tipo declarado en el encabezado (Header always set X-Content-Type-Options "nosniff"), en el Content-Type. Evita que se meta en un <iframe> (Header always set X-Frame-Options "DENY") y activa el filtro XSS (Header always set X-XSS-Protection "1; mode=block").

Ejemplo en .htaccess en /app:

```
<IfModule mod_headers.c>  
  Header always set X-Content-Type-Options "nosniff"  
  Header always set X-Frame-Options "DENY"  
  Header always set X-XSS-Protection "1; mode=block"  
</IfModule>
```

Ejemplo en docker-compose.yml:

command: bash -c "a2enmod headers"

Comentarios sospechosos

Para evitar la divulgación de información en comentarios, es recomendable eliminar información sensible de los comentarios. En este caso se han eliminado comentarios que contenían *TODOs*

Explicación:

Los *TODOs* son recursos que usan los programadores para identificar elementos de código que faltan por hacer o implementar. Esto puede ser de gran utilidad para los atacantes, ya que les hace saber que elementos, posiblemente de seguridad, no están presentes todavía en el código. Eliminando estos *TODOs*, se le priva de esta información al atacante, añadiendo una pequeña capa de seguridad extra.

Ataques por fuerza bruta

La manera más simple de evitar que se obtengan contraseñas mediante fuerza bruta es forzar a los usuarios a que tengan contraseñas más complejas; incluyendo, como mínimo, una mayúscula, una minúscula, un número y un carácter especial, además de una longitud mínima.

Explicación:

En la versión actualizada, a la hora de registrarse, el sistema obliga al usuario a seguir las restricciones previamente mencionadas al momento de definir una contraseña. El uso de estas restricciones hace que las contraseñas de los usuarios sean más complejas y, por ende, más difíciles de adivinar.

Ocultar cabecera “Server”

Explicación:

Para ocultar la cabecera “Server”, se debe editar el archivo `security.conf` y modificar dos líneas para evitar la divulgación de la versión del servidor web, así como el sistema operativo en el que está situado.

- `ServerTokens`: Esta directiva controla si el campo `Server` de las cabeceras de las respuestas que se envían de vuelta a los clientes incluye una descripción del sistema operativo genérico del servidor así como información sobre los módulos compilados en el servidor. Existen cinco categorías `Full` | `OS` | `Minimal` | `Minor` | `Major` | `Prod`; siendo `Full` la que más información concede y `Prod` la que menos.

De manera predeterminada está en `OS`, mostrando tanto el entorno de la web como la versión. Para ocultar los datos, se recomienda modificar el parámetro a `Prod`, de tal manera solo se muestra que la web está estructurada en Apache, la información mínima.

- `ServerSignature`: Por defecto, aparece en `Off`. Si se usa el valor `On`, se añade una línea (pie de página) con el número de versión y el valor de `ServerName`. Para ocultar la información se debe mantener en `Off`, que suprime la generación del pie de página, no mostrando ninguna información.

Donde colocar las líneas:

El fichero `docker-compose.yml` es un fichero en formato `YAML` que contiene las instrucciones para crear y configurar los servicios que van a constituir la aplicación o entorno.

En su estructura, dentro de `service`, `web`, está `command`. Esta sección permite ejecutar comandos en el `bash` del `docker` cuando se crea el entorno. Los comando que se ejecutarán son:

```
sed -i 's/ServerTokens OS/ServerTokens Prod/g' /etc/apache2/conf-available/security.conf
sed -i 's/ServerSignature On/ServerSignature Off/g' /etc/apache2/conf-available/security.conf
```

Estos dos comandos, modifican las dos líneas explicadas previamente para ocultar la cabecera “Server”. El comando `sed` de Linux te permite buscar, sustituir, insertar y eliminar líneas en un archivo sin abrirlo con un editor de texto.

Bibliografía

- Rick-Anderson. (s. f.). Prevención de ataques de falsificación de solicitudes entre sitios (CSRF) en ASP.NET MVC. Microsoft Learn. <https://learn.microsoft.com/es-es/aspnet/web-api/overview/security/preventing-cross-site-request-forgery-csrf-attacks>
- X-Powered-By: Castor. (2010, 18 abril). Removing X-Powered-By. Stack Overflow. <https://stackoverflow.com/questions/2661799/removing-x-powered-by>
- Astrahan, J. (2017, 17 enero). Docker-Compose won't volume my php.ini file. Stack Overflow. <https://stackoverflow.com/questions/41691584/docker-compose-wont-volume-my-php-ini-file>
- Clickjacking Defense - OWASP Cheat Sheet Series. (s. f.). https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html
- X-Frame-Options header - HTTP | MDN. (s. f.). <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options>
- HTTP cookies - HTTP | MDN. (s. f.). <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Cookies>
- Kevin. (2021, 30 junio). Samesite y cookies de sesion. Stack Overflow En Español. <https://es.stackoverflow.com/questions/465970/samesite-y-cookies-de-sesion>
- Content Security Policy (CSP) - HTTP | MDN. (s. f.). <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/CSP>
- CentralCSP. (s. f.). CentralCSP. <https://centralcsp.com/articles/how-to-build-the-best-csp>

- Rahman, A. (2022, 23 febrero). Content Security Policy with PHP. DEV Community. <https://dev.to/devlobb/content-security-policy-with-php-5d07>
- Apache HTTP. (s.f.). Configuración global del servidor - Servidor HTTP Apache. <https://apache2.com/2.0.58/server-wide.html>
- Hostinger. (2025, 26 marzo). Cómo utilizar el comando sed de Linux en 2025: ejemplos útiles y casos de uso. <https://www.hostinger.com/es/tutoriales/comando-sed-linux>
- Araujo, A., & Araujo, A. (2025, 28 octubre). ¿Qué es una Cross Site Request Forgery (CSRF) y cómo se soluciona? Hackmetrix Blog. <https://blog.hackmetrix.com/csrf-cross-site-request-forgery/>
- Cross Site Request Forgery (CSRF) | OWASP Foundation. (s. f.). <https://owasp.org/www-community/attacks/csrf>